BVS1 – Betriebssysteme

Lecture 01 – Introduction · Wintersemester 2026/2027

Stavros Kousidis

TH Köln · ICCT

Organization

How We Work in BVS1

- **Ask questions** and challenge claims.
- **Discuss and collaborate** while learning.
- Treat mistakes and failed attempts as **learning opportunities**.
- Respect your fellow students and their learning.
- Please arrive on time; if you are late, enter quietly.

Collaboration is encouraged while learning; demonstrated understanding is individual.

Our Learning Goal

By the end of BVS1, you should be able to:

motivate fundamental operating-system concepts from the requirements of a computer system,
explain how they interact, and
implement them in a minimal operating system that provides a simple shell as its user interface.

motivate $\longrightarrow$ **explain** $\longrightarrow$ **implement**

How BVS1 Works

Lecture — develop why

↓

Exercise — investigate and reason

↓

Lab — work toward the current system milestone

↓

Checkpoint — demonstrate and explain

↓

Electronic exam — transfer conceptual understanding

You need **basic programming** and a basic machine model.

We refresh **C** as needed and introduce the required **RISC-V assembly** along the way.

No prior OS-development experience is required.

Weekly Schedule and Support

	Day	Time	Room
Lecture	Friday	08:50–10:20	**ZW6-1**
Exercise	Friday	10:35–12:20	**ZW6-1**
Lab	Wednesday	08:50–12:20	**ZO8-6**
Lab	Friday	14:05–17:20	**ZO8-6**

The lab takes place every week.

Only the three checkpoint appointments are mandatory; all other lab sessions are optional, but we are present to help.

Teaching team: Stavros Kousidis · Raphael Ahrens · Anas Jabaly · Abderrahim Benlalam

Lab and Mandatory Checkpoints

Group	Checkpoint 1	Checkpoint 2	Checkpoint 3
M1	Wed **04 Nov**	Wed **09 Dec**	Wed **20 Jan**
M2	Wed **18 Nov**	Wed **16 Dec**	Wed **13 Jan**
F1	Fri **06 Nov**	Fri **11 Dec**	Fri **15 Jan**
F2	Fri **13 Nov**	Fri **18 Dec**	Fri **22 Jan**

M = Wednesday · F = Friday

Work individually or in pairs.

In a pair, both students must demonstrate individually that they meet the checkpoint requirements.

Checkpoint Rules

- Book checkpoint appointments through **ILU**.
- Changes require a **justified case**; retakes are also organised through ILU.
- A failed checkpoint can be **retaken**.
- A yellow card is recorded for a **failed attempt, late arrival, or no-show**.
- If you expect a delay, email bvs1@f07.th-koeln.de **before the mandatory appointment**.

Three yellow cards → lab not passed → no exam admission.

Assessment

Lab

- ungraded,
- three checkpoints must be passed,
- passing the lab is required for **exam admission**.

Electronic exam

- graded,
- **closed-book, but self-contained**,
- no personal notes, books, scripts or electronic material,
- non-core technical information needed to solve a task is provided with the task.

The exam tests understanding and transfer, not lookup of terminology.

Help Us Improve BVS1

BVS1 is being developed iteratively.

Your constructive feedback helps us improve the **lecture, exercises and lab** — for you and for future students.

```
your feedback
      ↓
we analyse it
      ↓
we change what should improve
      ↓
we tell you what changed
```

Feedback is welcome throughout the semester; we will also ask for it explicitly at selected points.

Operating Systems

What Are You Operating?

Particify P1 · Multiple choice

Which operating-system environments do you currently use?

Select all that apply.

- Windows
- macOS
- Linux-based systems
- Android
- iOS / iPadOS
- ChromeOS
- Other
- I am not sure



partici.fi/17502793

Which of These Are Operating Systems?

Particify P2 · Multiple choice



partici.fi/17502793

Which of these would you currently call an operating system?

Select all that apply.

- Windows
- Ubuntu
- Linux kernel
- Android
- UEFI
- QEMU
- Docker
- Firefox
- the thing that makes the computer go brrrrr

Prediction

Particify P3 · Prediction



partici.fi/17502793

Can a computer execute one program without an operating system?

1. Yes
2. No
3. I am not sure

Remember your answer.

Experiment 1 — Hello BVS1

We give a machine one tiny program.

Watch what happens.

```
Hello BVS1
```

What must have happened before these characters appeared?

Particify P4 · Think



partici.fi/17502793

Which statements are true?

- QEMU executed our program.
- A virtual CPU executed our program.
- Ubuntu executed our guest program.
- The guest had an operating system.
- QEMU was already running before our guest program started.
- `Hello BVS1` existed somewhere in memory while the program ran.

Vote individually. Then compare your answers in pairs for two minutes.

Where do you disagree? What supports your choice?

Observation

Our guest program ran.

The target machine had **no guest operating system**.

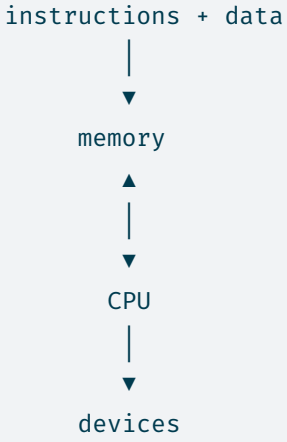
Therefore:

> *Executing instructions is not, by itself, what makes an operating system necessary.*

What Was Required?

One important idea of the **stored-program / Von Neumann model**:

instructions and data are both kept in memory



How Can Software Know What a CPU Understands?

Different processor implementations can run the same machine code only if they agree on a contract.

ISA — Instruction Set Architecture

Among other things, an ISA specifies:

- instructions and encodings,
- registers,
- memory-addressing rules,
- exceptional control transfers,
- privilege mechanisms.

For BVS1 we use **RV64: 64-bit RISC-V**.

Your Development Machine Is Not Our Target

Your Ubuntu development VM may use **x86-64 or Arm64**.

Our teaching target uses **RV64**.

So there are two different questions:

1. Where do our development tools run?
2. Which machine are we building the program for?

Development host ≠ target machine

Virtualization vs. Emulation

Can the physical CPU execute the guest / target instructions directly?

Virtualization

same ISA

```
guest instruction
       ↓
physical CPU
```

Mostly **direct execution**.

Virtualization controls privileged access.

BVS1: Multipass → Ubuntu VM

Emulation

different ISA possible

```
RV64 instruction
       ↓
QEMU
       ↓
host operation
```

QEMU **interprets or translates**.

BVS1: QEMU → RV64 target

Our Actual Machine Stack

```
physical host
     ↓  virtualization (Multipass)
Ubuntu development VM: bvs1
     ↓  runs
QEMU process
     ↓  emulates RV64
RV64 target machine
     ↓  executes
Hello BVS1
```

Only the RV64 target is our teaching machine.

It contains **no guest operating system**.

The Toolchain Connects Development Host and Target

```
source code
    ↓
cross compiler / assembler
    ↓
object files
    ↓
linker
    ↓
RV64 executable
    ↓
QEMU
    ↓
RV64 target machine
```

Change the Problem

Our machine now needs to run **two programs**.

What new problems appear?

Write down as many as you can.

We will collect them on the board.

Increase the Requirements

Now add:

- many programs,
- multiple users,
- persistent storage,
- keyboard and display,
- network,
- failures,
- programs that do not trust each other.

What responsibilities would you want software to take over?

Our Working Definition

An **operating system** is software that helps turn hardware into a usable computer system by providing:

- abstractions,
- resource management,
- coordination,
- protection,
- services for programs and users.

We will test and sharpen this definition throughout the semester.

What Must We Add to Hello BVS1?

Particify P5 · Open question

What must we add to `Hello BVS1`
before you would call the result an operating system?



partici.fi/17502793

```
Hello BVS1
    ↓
    ?
    ↓
operating system
```

Keep your ideas.

We will return to them during the semester.

What We Are Going to Build

A small educational operating system on:

RV64 + QEMU `virt`

We start with:

```
Hello BVS1
```

Then add mechanisms **only when a requirement motivates them**.

The long-term target is a **simple shell**.

Before You Leave

Challenge these claims:

1. A computer needs an operating system to execute instructions.
2. Virtualization and emulation mean the same thing.
3. An operating system is mainly a graphical user interface.
4. “Operating system” and “processor architecture” describe the same layer.

For each claim:

agree, disagree, or qualify — and why?

References

Selected references:

- Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*
- Tanenbaum & Bos, *Modern Operating Systems*
- RISC-V ISA specifications
- QEMU RISC-V `virt` documentation
- Canonical Multipass documentation

Take-Home Concepts

If you remember five things from today:

- A CPU can execute a program **without a guest operating system**.
- The **ISA** is the machine-level contract between software and processor.
- **Development host** and **target machine** are different roles.
- **Virtualization** and **emulation** solve different problems.
- An **operating system** becomes necessary as requirements for sharing, abstraction, coordination, protection, and services grow.
  We start with a machine that runs one program — and add OS mechanisms only when a requirement demands them.